

Automated Data Pipeline

Week 3 - Task 2 Documentation

Python Programming Internship - Aptura Tech Solutions

12	7	5	24/24
Input Records	Valid Records	Invalid Records	Automated Tests

1. Project Overview

Objective. Build a practical Python pipeline that reads raw CSV or JSON data, validates and cleans records, transforms selected fields, generates summary statistics, and exports a clean dataset together with an error log.

Chosen dataset. A small sales-order dataset was used because it provides clear numeric, text, date, duplicate-ID, and validation scenarios without introducing dependencies outside the assignment scope.

2. Approach and Design Decisions

- Use one reusable DataPipeline class so the same workflow can process CSV and JSON inputs.
- Keep the implementation dependency-free by using Python standard-library modules such as csv, json, pathlib, datetime, re, collections, and unittest.
- Clean fixable values before validation so imperfect formatting is corrected instead of automatically rejected.
- Separate valid and invalid records: valid records continue to the clean dataset; invalid records are preserved in an error log with the reason and original raw record.
- Calculate outputs and summary statistics from processed data instead of hard-coding results.

Pipeline workflow

Read	Clean	Validate	Transform	Separate	Summarize	Export
------	-------	----------	-----------	----------	-----------	--------

3. Validation, Cleaning, and Transformations

Field	Validation	Cleaning / Transformation	Example
record_id	Required and unique	Trim whitespace	duplicate 1002 -> error
customer_name	Required	Trim + title case	Sara Ahmed -> Sara Ahmed
email	Required + email format	Trim + lowercase	ALI.KHAN@... -> ali.khan@...
city	Required	Trim + title case	KARACHI -> Karachi
order_date	Valid date	Normalize accepted formats	2026/09/04 -> 2026-09-04
quantity	Positive whole number	Trim + integer conversion	3 -> 3
unit_price	Number greater than 0	Trim + float conversion	1750.50 -> 1750.50
total_amount	Derived from valid record	quantity x unit_price	3 x 1750.50 = 5251.50

4. Generated Outputs

Output	Purpose
cleaned_data.csv	Contains only valid, cleaned, transformed records with total_amount.
error_log.csv	Contains rejected records, record number, record ID, validation reason, and original raw record.
summary.json	Contains measurable processing statistics and per-city record counts.

5. Testing and Measurable Results

Metric	Verified Result
Input records	12
Valid records	7
Invalid records	5
Valid rate	58.33%
Total sales amount	38,751.50
Average order value	5,535.93
Minimum order value	4,500.00

Metric	Verified Result
Maximum order value	8,000.00
Automated tests	24 passed / 24 run
Input parity	CSV and JSON produced the same logical summary

Rejected-record reasons verified in the error log: invalid email format, zero quantity, negative unit price, duplicate record ID, and missing customer name.

Automated test coverage includes: CSV/JSON readers, missing/unsupported inputs, cleaning functions, validation rules, duplicate detection, transformations, error routing, summary statistics, export generation, and complete CSV/JSON pipeline runs.

6. Evidence

The following screenshots are stored in the project evidence folder:

- 01_project_structure.png
- 02_csv_pipeline_summary.png
- 03_json_pipeline_summary.png
- 04_cleaned_dataset.png
- 05_error_log.png
- 06_automated_tests_24_passed.png

7. How to Run

`python main.py` - Run the default CSV pipeline.

`python main.py .\data\raw_data.json` - Run the same pipeline with JSON input.

`python .\tests\test_pipeline.py -v` - Run the automated test suite.

8. Reflection and Next Steps

Reflection. The main learning from this task was designing a repeatable data-processing flow where cleaning and validation are separate decisions. Fixable formatting issues are normalized, while genuinely invalid records are rejected with traceable reasons instead of being silently discarded.

Possible next steps. Add configurable schemas and validation rules, structured application logging, support for larger files through streaming/chunked processing, additional output formats, and broader edge-case tests.

Task 2 documentation - concise implementation, testing, results, and reflection